

Week 8 — Lab Handout (Student)

Error Handling & Feedback

Time: ~90 min in-class **Work as:** Your team (5 members) **AI use this week:** ✓ **ON — required.** Scaffold, review, and **log your prompts.** **Due at end of session:** Deliverable 3 (Interface & View Binding, 30%)

Why this lab matters

Your app works when everything goes right. Today you make it work when things go *wrong* — which, in the real world, is often. Every action gets a loading state, every failure gets a visible, helpful message, and the whole app starts behaving consistently. This is the step that turns your project from a demo into something a person could actually use.

Task 1 — Build a feedback matrix (15 min)

For every action in your app (create, read, edit, delete, load), write down its three states.

Action	Loading	Success	Error
Create order	"Saving..." spinner	row appears + toast	422 field errors
Delete order	button disabled	row removed + toast	500 → "Couldn't delete"
Load list	skeleton/spinner	rows render	network → retry

- ☐ Every action has a loading, success, and error entry.
- ☐ Save it to `/docs/feedback-matrix.md`.

Task 2 — Add loading indicators everywhere (15 min)

- ☐ Every async action shows a loading state (spinner, skeleton, or a disabled control labelled "Saving...").
- ☐ Controls are **disabled while pending** (no double-submits).
- ☐ The list has a loading state before data arrives.

Task 3 — Handle every failure category (25 min)

Wire the branches your controllers can actually return — not just success.

- ☐ **422 validation** → show the field-level errors **inline**, next to the fields.
- ☐ **404 not found** → show a clear "not found" state, not a blank screen.
- ☐ **500 server / network** → show a general "something went wrong" message with a way to retry.
- ☐ Add a **confirmation** step before destructive actions (delete).

If breaking the network or submitting bad data shows *nothing*, your error handling is missing. Every failure must be visible.

Task 4 — Make messages helpful (10 min)

Rewrite any vague or technical message so it's specific, actionable, and in your app's voice.

- ☐ No raw status codes or stack traces reach the user.
- ☐ Field errors name the field and the fix ("Quantity must be at least 1.").
- ☐ Global messages say what happened and what to do.

Task 5 — Make it consistent (10 min)

- ☐ Loading, success, and error look and behave the **same** across every screen.
- ☐ Ideally, feedback comes from one shared component/helper reused everywhere.

Task 6 — Test the failure paths, then ship (15 min)

Deliberately try to break it:

- ☐ Submit invalid data → inline 422 errors appear.
- ☐ Request a record that doesn't exist → not-found state.

- ☐ Simulate a failed/slow request → general error + retry, UI never freezes.
 - ☐ Record results in `/docs/feedback-tests.md`.
 - ☐ Log prompts in `/docs/ai-notes/week-08.md`; merges via reviewed PR.
-

Deliverable 3 — Final Submission Checklist

Confirm the full submission (see the Deliverable 3 spec for the Definition of Done):

- ☐ A fully interactive local app: create, read, update, delete all work end to end
 - ☐ Every screen and **state** built (list, detail, create, edit, empty, loading, error)
 - ☐ Loading, success, and error handled on every action; failures always visible
 - ☐ Messages are helpful, human, and consistent
 - ☐ Prompt log present, labelling AI-generated vs. hand-written
 - ☐ Every member owns work on the board; all merges via reviewed PR
-

Looking ahead to Phase 4 (Week 9)

AI becomes the subject. You'll turn your review skills on AI-generated code — finding the subtle bugs and security holes it introduces — in a code-review week where no code merges without a peer sign-off. Everything you built these three weeks is what makes you able to judge it.